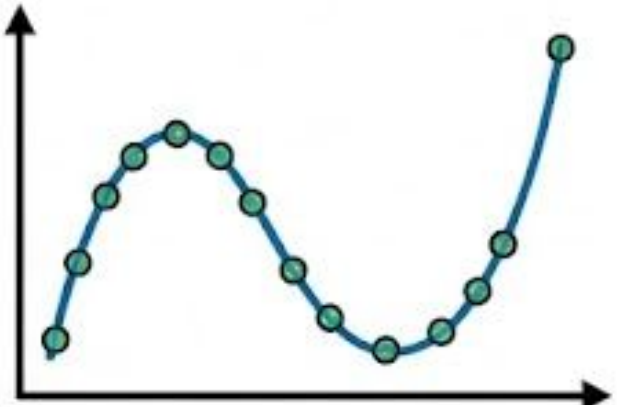
Sistemas Inteligentes

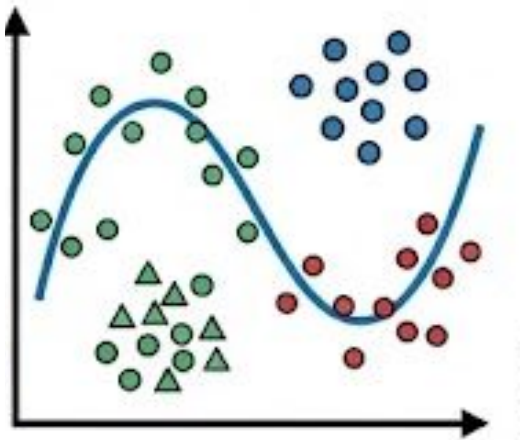
Profa. Dra. Nathalia da Cruz Alves

Treinamento de Modelos

Por que um modelo com 99% de acurácia no conjunto de treino pode falhar no mundo real?



Acurácia no conjunto de treino: 99%



Acurácia em produção: 50%

Treinamento de Modelos

Um modelo pode ter 99% de acurácia no conjunto de treino e ser **inutilizável** no mundo real

Treino ≠ realidade

É necessário testar o modelo com “dados nunca vistos”

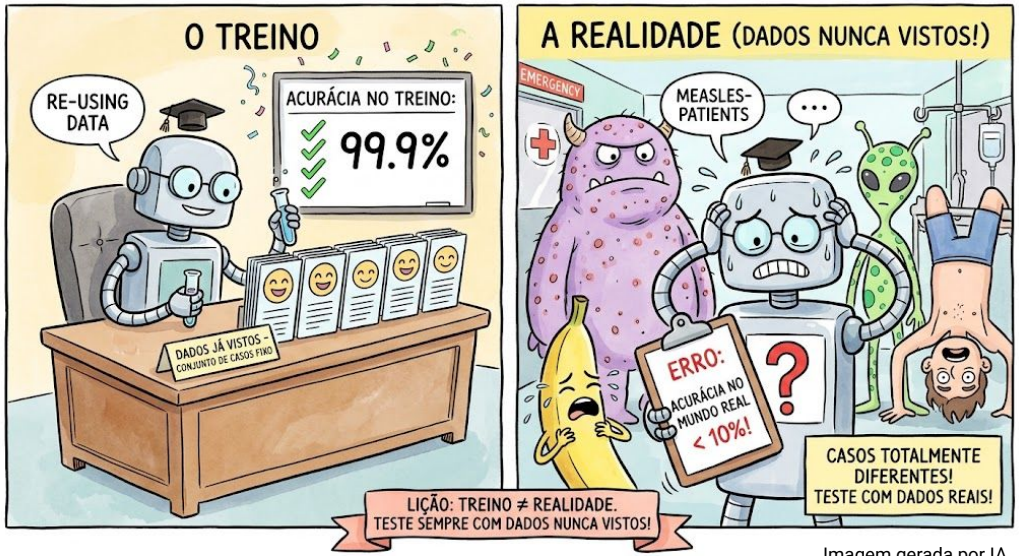


Imagem gerada por IA.

Sem dados nunca vistos, seria como treinar um médico apenas com um único conjunto de casos e depois testá-lo com os mesmos pacientes

Separação do Conjunto de Dados

A separação do conjunto de dados é **essencial**

Normalmente separamos em:

Dados para TREINO

Objetivo: Aprender padrões

Dados para VALIDAÇÃO

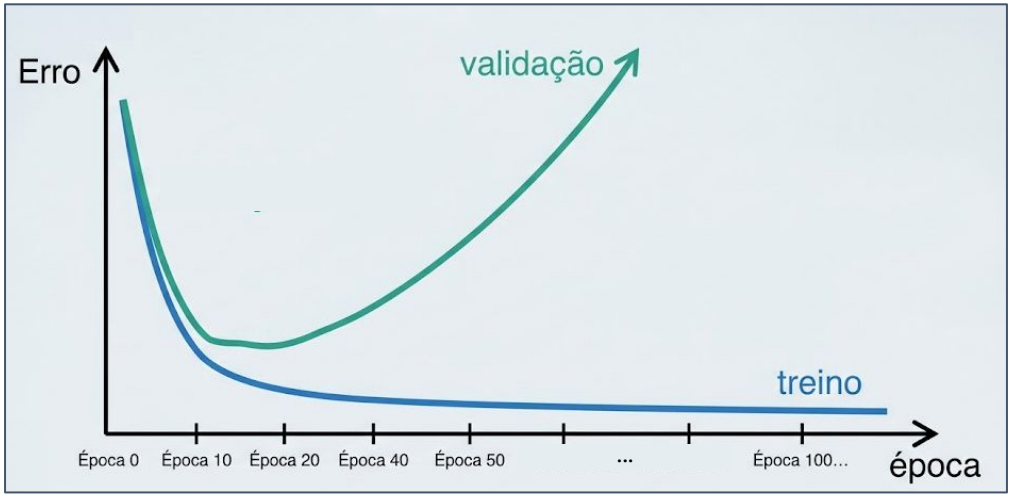
Objetivo: Ajustar decisões

Dados para TESTE

Objetivo: Simular mundo real

Comportamento do Erro

- No começo do treinamento, o erro é **alto** no conjunto de **treino e validação**
- Conforme o modelo aprende, o erro tende a **diminuir** no conjunto de **treino**
- Porém, no conjunto de **validação**, o erro pode começar a **aumentar**



Comportamento do Erro

- No começo do treinamento, o erro é **alto** no conjunto de **treino e validação**
- Conforme o modelo aprende, o erro tende a **diminuir** no conjunto de **treino**
- Porém, no conjunto de **validação**, o erro pode começar a **aumentar**



Por quê?

Viés-Variância (*Bias-Variance*)

Bias-Variance é uma decomposição teórica do **erro de generalização** na teoria de aprendizado estatístico

$$\mathbb{E}\left[(y-\hat{f}(x))^2\right]=\mathrm{Bias}^2+\mathrm{Variance}+\mathrm{Noise}$$

Bias (Viés)

- Modelo **rígido/simples demais**
- Não consegue capturar os padrões reais dos dados

Variance (Variância)

- Modelo **sensível demais** aos dados de treino
- Pequenas mudanças nos dados geram grandes mudanças no modelo

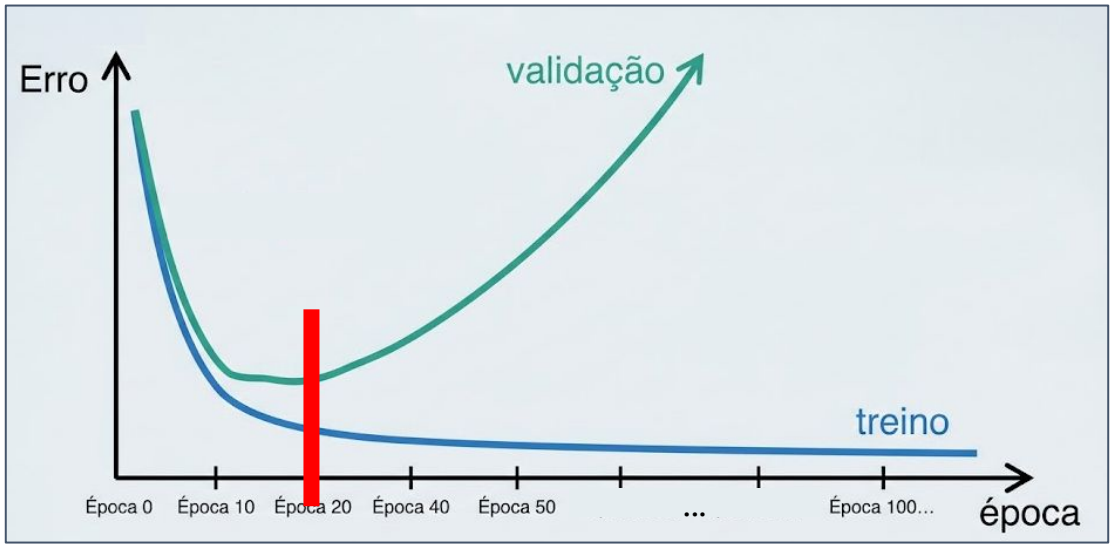
Noise (Ruído)

- Parte aleatória e **irredutível** do problema
- Presente nos próprios dados

Bias-Variance

Existe um *trade-off* entre *bias* e variância

- Em muitos casos, reduzir um **aumenta** o outro



Como encontrar o equilíbrio entre **aprender pouco** e **aprender demais**?

Conceitos importantes no treinamento de modelos:

- Subajuste - *Underfitting*
- Sobreajuste - *Overfitting*
- Validação Cruzada
- Hiperparâmetros

Intuição *Underfitting* vs. *Overfitting*

Vamos supor que tenhamos dois desafios em vista:

Atravessar um deserto de milhares de quilômetros



Buscar pão na padaria da esquina



Intuição *Underfitting* vs. *Overfitting*

Para executar tais tarefas temos disponível:

Jeep *off-road*



Patinete elétrico



Intuição *Underfitting* vs. *Overfitting*

Embora a pergunta possa parecer trivial, um projeto de *Machine Learning* mal conduzido pode resultar na escolha de um **modelo inadequado** para o problema

Overfitting

- Escolher o Jeep *off-road* para ir na padaria

Underfitting

- Escolher o patinete elétrico para atravessar o deserto

Definição de *Overfitting*

Ocorre quando o modelo aprende ***excessivamente*** *os padrões específicos, incluindo ruídos e particularidades, do conjunto de treinamento*

- Baixo erro nos dados de treino
- Alto erro em dados não vistos
- Baixa capacidade de generalização
- Associado a alta variância

Consequência: o modelo memoriza os exemplos de treinamento em vez de aprender padrões gerais do problema

Definição de *Underfitting*

*Ocorre quando o modelo possui complexidade **insuficiente** para representar adequadamente os padrões existentes nos dados*

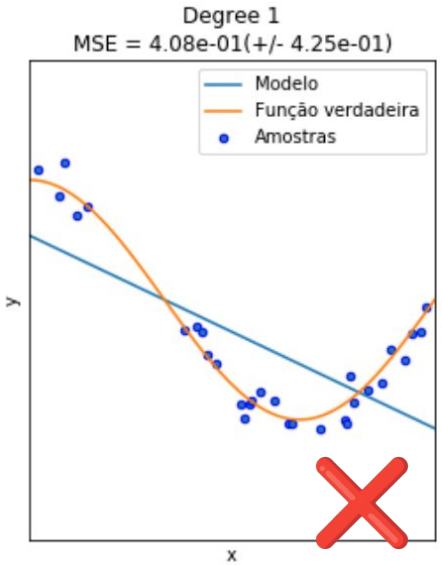
- Alto erro nos dados de treino
- Alto erro em dados de teste
- Incapacidade de capturar relações relevantes entre as variáveis
- Associado a alto *bias*

Consequência: o modelo não aprende adequadamente a estrutura do problema, resultando em desempenho insatisfatório

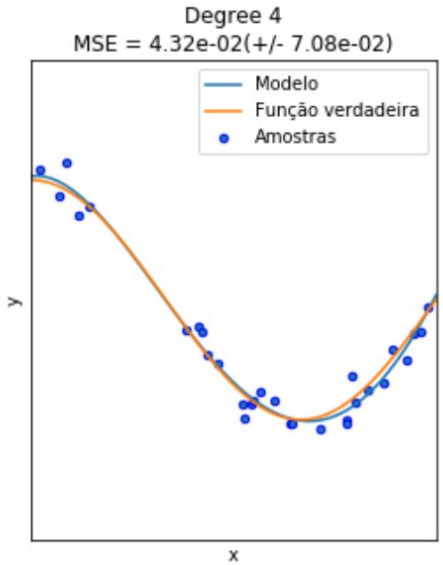
Exemplo de *Underfitting* vs. *Overfitting*

Encontrar uma função que aproxima melhor os dados

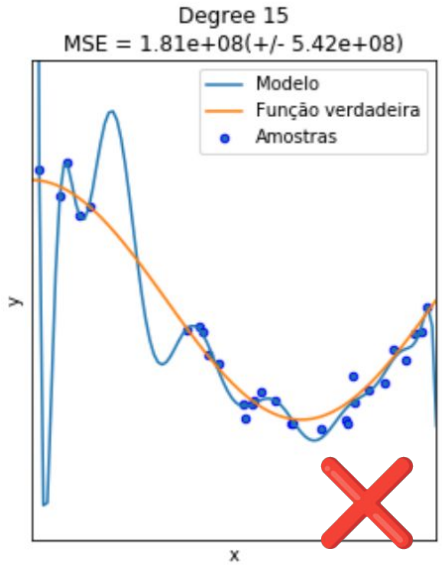
Modelo linear



Polinômio de grau 4



Polinômio de grau 15



Exemplo de *Underfitting* vs. *Overfitting*

Underfitting

- Modelo linear é simples demais para aproximar a função

Overfitting

- Polinômio de grau 15 é muito sensível a ruído e não consegue generalizar bem o problema

Avaliação de Modelos

A avaliação de modelos deve ser conduzida de forma criteriosa para garantir que os resultados reflitam a capacidade real de **generalização** do modelo

- O conjunto de teste deve permanecer **completamente isolado** durante o desenvolvimento do modelo
- **Não** se deve
  - Utilizar o conjunto de teste para ajustar hiperparâmetros
  - Comparar repetidamente modelos com base nesse conjunto
- Caso contrário, o processo passa a incorporar informações do teste ao treinamento, produzindo estimativas **otimistas** e **enviesadas** de desempenho

Por que não avaliar o modelo com os dados de treinamento?

Treinar e avaliar um modelo utilizando **as mesmas amostras** constitui um **erro metodológico**

- Nesse cenário, o modelo pode simplesmente memorizar os dados observados, obtendo desempenho artificialmente elevado sem necessariamente aprender padrões generalizáveis

Objetivo central:

- Avaliar a capacidade do modelo de realizar **previsões corretas em dados não vistos** anteriormente

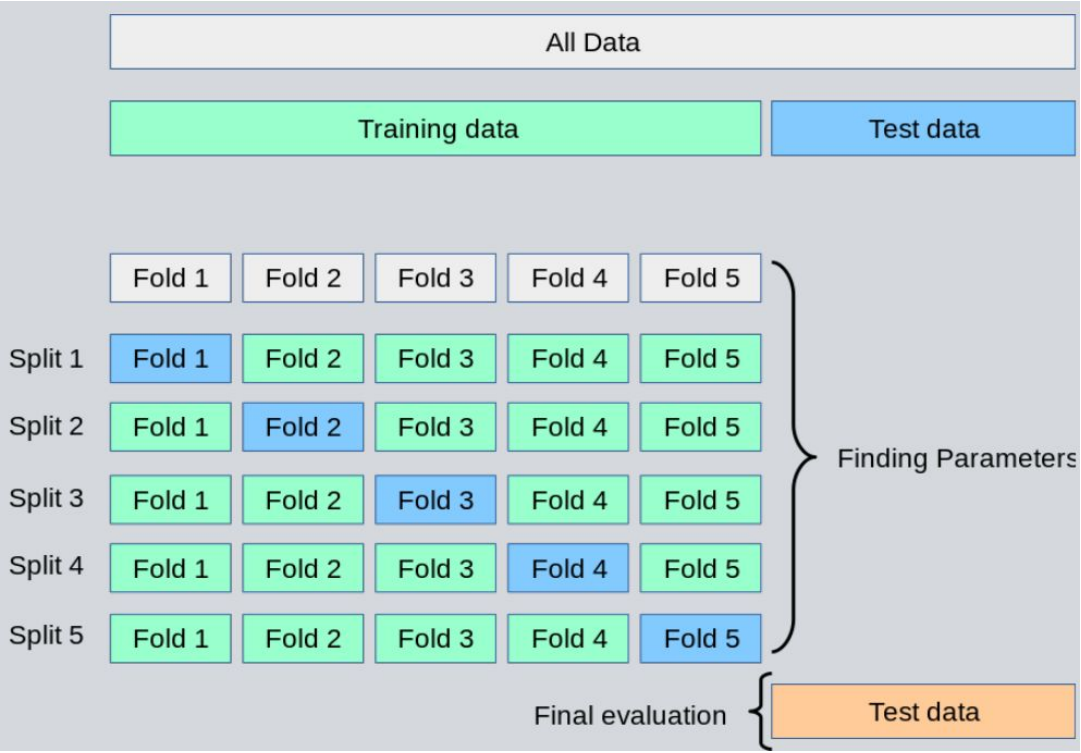
Motivação para Validação Cruzada

Para estimar o desempenho do modelo de forma mais confiável, sem utilizar o conjunto de teste durante o treinamento, podem ser empregadas **técnicas de validação**, como a Validação Cruzada (*Cross-Validation*)

Validação Cruzada

- Dividimos o conjunto de treinamento em *k* partes (*folds*)
- Para cada uma das *k* partes:
  - Treinamos o modelo usando os $k - 1$ *folds* restantes
  - Avaliamos o modelo no *fold* separado, chamado de conjunto de validação
- Repetimos o processo *k* vezes, alternando o *fold* de validação
- A métrica final é a média dos desempenhos obtidos em cada iteração

Validação Cruzada



Validação Cruzada

Caso especial: *Leave-One-Out Cross-Validation* (LOOCV)

- Quando k = n (número de amostras)
  - Cada iteração utiliza
    - 1 amostra para validação
    - n − 1 amostras para treino

Controle de complexidade do modelo

Mesmo utilizando técnicas de validação, um modelo ainda pode apresentar baixa capacidade de generalização se a sua **complexidade** não for controlada adequadamente

O que fazer?

Regularização

A regularização consiste em um conjunto de técnicas utilizadas para **controlar a complexidade** do modelo durante o treinamento

O principal objetivo é

- Reduzir o risco de *overfitting*
- Melhorar a capacidade de generalização
- Favorecer modelos mais robustos e estáveis

Intuição: um modelo muito livre aprende até o ruído dos dados, e não só o padrão

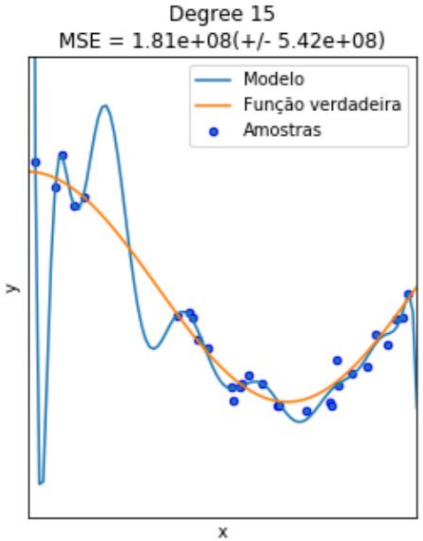
Regularização

Sem regularização:

- O modelo fica “livre demais”
- Aprende detalhes inúteis
- Vai muito **bem no treino e mal no teste**

Com regularização:

- O modelo é “freado”
- Precisa aprender padrões mais gerais
- **Generaliza** melhor para dados novos



Tipos comuns de regularização

Dropout

- Desliga neurônios aleatoriamente durante o treino em uma Rede Neural
- Força a rede a não depender de um único neurônio

L2 (*weight decay*)

- Penaliza pesos muito grandes
- Evita que o modelo fique extremamente confiante

Técnicas com efeito regularizador

Batch Normalization

- Técnica de estabilização do treinamento
- Normaliza as ativações de uma camada durante o treinamento para deixá-las com distribuição mais estável (média ≈ 0, variância ≈ 1)

Early Stopping

- Para o treino quando o modelo começa a piorar na validação
- Intuição: evita treinar demais

Hiperparâmetros

Modelos de Machine Learning **não são definidos apenas pelos dados**

Eles também dependem de **escolhas feitas antes do treino,** chamadas de **hiperparâmetros**

Exemplos de hiperparâmetros

- Número de camadas da rede
- Número de neurônios por camada
- Taxa de aprendizado (*learning rate*)
- Função de ativação
- Otimizador



Imagem gerada por IA.

Como encontrar os melhores hiperparâmetros?

Não sabemos de antemão qual combinação funciona melhor

- Muitas combinações possíveis
- Testar manualmente é lento e ineficiente
- Pequenas mudanças podem afetar muito o desempenho

Busca de hiperparâmetros

Algoritmos ou estruturas de experimentação que testam diferentes configurações de modelos e encontram a melhor com base em validação e configurações testadas

Ideia geral:

- É um processo de busca
- Uma camada acima do treinamento
- Um otimizador de configurações de modelos

Não aprende os parâmetros do modelo, apenas seleciona hiperparâmetros e configurações do processo de treinamento

Busca em Grade (*Grid Search*)

*Método de otimização de hiperparâmetros que realiza uma busca **exaustiva**, testando todas as combinações possíveis dentro de um espaço de busca previamente definido*

Ideia geral

- Define um conjunto de valores possíveis
- Treina um modelo para cada combinação
- Avalia cada modelo em validação
- Seleciona a melhor configuração **dentre as combinações testadas**

Limitações

- Crescimento exponencial de combinações
- Alto custo computacional
- Nem sempre é viável testar tudo

Busca Aleatória *(Random Search)*

*Método de otimização de hiperparâmetros que seleciona **aleatoriamente** combinações de valores dentro de um espaço de busca previamente definido*

Ideia geral

- Define um conjunto de valores possíveis
- Treina um modelo para cada combinação selecionada aleatoriamente
- Avalia cada modelo em validação
- Seleciona a melhor configuração **dentre as combinações testadas aleatoriamente**

Limitações

- Não garante encontrar a melhor combinação possível
- Os resultados dependem das combinações sorteadas

Em geral...

- Quando um modelo falha em produção, quase nunca é porque ele não conseguiu aprender os dados de treino

- O problema é que o modelo:
  - **Aprendeu demais**
  - **Aprendeu de menos**
  - Foi **mal avaliado**

- Um bom modelo não é o que apenas acerta o que já viu, mas o que também acerta o que nunca viu

Em geral...

Todos os conceitos vistos (*overfitting*, *underfitting*, *bias*, variância, validação cruzada, regularização e busca de hiperparâmetros) são **ferramentas** para reduzir **erro de generalização**

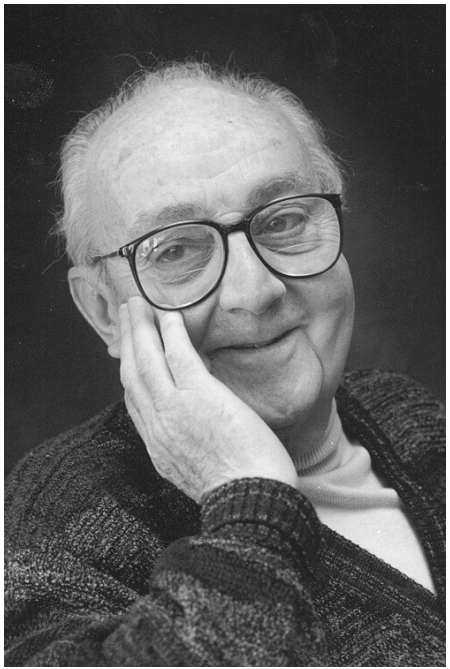
Exercícios

1. Durante o treinamento de uma rede neural, um modelo apresenta ótimo desempenho nos dados de treinamento, mas resultados significativamente piores nos dados de validação. Esse comportamento pode indicar *overfitting*. Explique o que é regularização no contexto de *Machine Learning*, como ela ajuda a reduzir o *overfitting* e cite duas técnicas de regularização utilizadas em redes neurais artificiais.

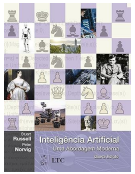
2. Um pesquisador deseja otimizar os hiperparâmetros de uma rede neural, mas possui recursos computacionais limitados. Ele considera utilizar *Random Search* em vez de *Grid Search*. Qual é a principal vantagem dessa escolha?

“Essentially, all models are wrong, … but some are useful”

George Box



Leia mais sobre este tema...



Russell, S., Norvig, Peter. Inteligência Artificial - Capítulo 18. 2ª Ed. Campus-Elsevier; Brasil, 2010.

Scikit-learn developers. Cross-validation: evaluating estimator performance. 2026. Disponível em: https://scikit-learn.org/stable/modules/cross_validation.html

Vargas Júnior, E. C. Treinando, testando e selecionando. Grupo de Estudos em Aprendizagem de Máquina / Departamento de Matemática / UFSC. 2020. Disponível em: https://geam.paginas.ufsc.br/files/2020/02/train-test-select.pdf

creative commons

COMMONS DEED

Atribuição-Uso Não-Comercial-Compartilhamento pela Licença 2.5 Brasil

Você pode:

- copiar, distribuir, exibir e executar a obra

- criar obras derivadas

Sob as seguintes condições:

Atribuição — Você deve dar crédito ao autor original, da forma especificada pelo autor ou licenciante.

Uso Não-Comercial — Você não pode utilizar esta obra com finalidades comerciais.

Compartilhamento pela mesma Licença — Se você alterar, transformar, ou criar outra obra com base nesta, você somente poderá distribuir a obra resultante sob uma licença idêntica a esta.

Para ver uma cópia desta licença, visite http://creativecommons.org/licenses/by-nc-sa/2.5/br/ ou mande uma carta para Creative Commons, 171 Second Street, Suite 300, San Francisco, California, 94105, USA.